



PDF Download
3658644.3691370.pdf
24 January 2026
Total Citations: 0
Total Downloads: 2345

Latest updates: <https://dl.acm.org/doi/10.1145/3658644.3691370>

POSTER

Poster: A Multi-step Approach for Classification of Malware Samples

ARNALDO SGUEGLIA, University of Sannio, Benevento, BN, Italy

ROCCO ADDABBO

ANDREA DI SORBO, University of Sannio, Benevento, BN, Italy

STANISLAV DASHEVSKYI, Forescout Technologies, Inc., San Jose, CA, United States

DANIEL RICARDO DOS SANTOS, Forescout Technologies, Inc., San Jose, CA, United States

CORRADO AARON VISAGGIO, University of Sannio, Benevento, BN, Italy

Open Access Support provided by:

University of Sannio

Forescout Technologies, Inc.

Published: 02 December 2024

[Citation in BibTeX format](#)

CCS '24: ACM SIGSAC Conference on
Computer and Communications Security
October 14 - 18, 2024
UT, Salt Lake City, USA

Conference Sponsors:
SIGSAC

Poster: A Multi-step Approach for Classification of Malware Samples

Arnaldo Sgueglia
University of Sannio
Benevento, Italy
sgueglia@unisannio.it

Rocco Addabbo
TIM
Rome, Italy
roccoaddabbo96@gmail.com

Andrea Di Sorbo
University of Sannio
Benevento, Italy
disorbo@unisannio.it

Stanislav Dashevskiy
Fore Scout
Eindhoven, Netherlands
stanislav.dashevskiy@fore Scout.com

Daniel dos Santos
Fore Scout
Eindhoven, Netherlands
daniel.dossantos@fore Scout.com

Corrado Aaron Visaggio
University of Sannio
Benevento, Italy
visaggio@unisannio.it

ABSTRACT

The rapid spread of unknown malware has prompted many companies and researchers to improve their detection and classification systems. Cyber security companies must deal with the newest malware samples captured by their honeypots, aiming to analyze and classify them to develop several countermeasures. This process could only be feasible with a strong ground truth baseline; companies could only securely store the samples, waiting for further developments. This paper proposes a multi-step approach to support the classification process of unknown malware samples. Specifically, our approach first leverages well-known classification techniques and third-party services to collect as much information as possible about the samples and combines them with Machine Learning (ML)-based techniques to classify the remaining samples. Our case study, conducted on industrial data, shows how the combination offers superior performance than using each method individually.

CCS CONCEPTS

• Security and privacy → Malware and its mitigation.

KEYWORDS

Malware Detection, Malware Classification, Threat Intelligence, Machine Learning, IoT

ACM Reference Format:

Arnaldo Sgueglia, Rocco Addabbo, Andrea Di Sorbo, Stanislav Dashevskiy, Daniel dos Santos, and Corrado Aaron Visaggio. 2024. Poster: A Multi-step Approach for Classification of Malware Samples. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, October 14–18, 2024, Salt Lake City, UT, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3658644.3691370>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CCS '24, October 14–18, 2024, Salt Lake City, UT, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0636-3/24/10

<https://doi.org/10.1145/3658644.3691370>

1 INTRODUCTION

Recently developed malware uses increasingly sophisticated techniques to evade companies' detection systems, and AI-driven malware creation is expected to increase risks further¹. Over time, malware attack patterns evolve, exhibiting multiple functionalities simultaneously and becoming more evasive and aware of monitored environments [3, 28]. Besides, most malware is not created from scratch but is almost always a variant of earlier malware [14]. Malware is considered unknown when it is unknown to the execution environment and exhibits a different behavior than other benign programs. According to recent reports, unknown malware is also rising, with over 450,000 new samples registered daily and over 100 million types detected in 2023². In 2022, over 465,000 unique malware variants were identified³. In addition, detecting and classifying unknown malware samples face several challenges, such as (i) relationship/dependencies among system entities, (ii) the lack of intrinsic distance/similarity metrics between two programs, and (iii) different program versions, which impacts on its signature [30].

In recent years, the unknown malware has prompted increasing studies in their detection and classification fields. Intrusion detection systems (IDS) approaches, such as signature, anomaly, and specification-based, have limitations in detecting zero-day attacks. IDS based on signatures yields a low false-positive rate [8], but attackers may readily alter the malware to avoid matching a signature [5]. Anomaly-based IDS is suitable for detecting zero-day attacks but has high false-positive ratios [7]. Specification-based IDS works similarly to anomaly-based ones but has limitations in defining attack detection policies and evaluating them in real-world scenarios [18]. Earlier authors investigated the unknown malware detection field [3, 21, 30] proposing techniques based on the frequency of invocation of opcode, learning the program representation and applying similarity metrics, or adapting Machine Learning network traffic algorithms to unknown malware.

Despite these limitations, researchers and companies must continuously deal with malware. Nowadays, companies have tools like honeypots to capture and analyze malware samples; in most cases, they are the first to deal with new malware samples. Without a strong ground truth baseline, they can only capture and securely

¹<https://www.crowdstrike.com/global-threat-report/>

²<https://www.av-test.org/en/statistics/malware/>

³<https://pixelprivacy.com/vpn/guides/guide-malware-statistics-and-facts/>

store the samples without extracting helpful information. Hybrid IDS is advanced to detect these attacks, combining several techniques to improve the overall performance. However, it requires human intervention due to the high false-positive ratios and complexity [22]. In contrast to earlier investigations, our work proposes a multi-step approach to support companies in classifying unknown malware samples. It leverages well-known classification techniques and integrates them with ML-based ones. Our case study, conducted in an industrial setting, highlights the feasibility of using the proposed approach to obtain improved classification performance.

2 METHODOLOGY

The approach consists of three sequential steps to classify unknown malware samples gradually. Each step is first evaluated on the ground truth using Precision, Recall, and Coverage metrics, and second, the best solutions are integrated into our pipeline and used to classify the samples in the dataset. Each step takes a set of malware samples as input and produces a subset of correctly classified samples; this subset goes directly to the classification aggregator, while the others follow the pipeline to other classification steps. Finally, after the third step, the unclassified malware samples undergo a manual classification phase involving security experts. For Hash Matching and Yara rules, **Precision** is the number of correct results divided by the number of returned results, **Recall** is the number of accurate results divided by the number of expected results, and **Coverage** is the number of matched samples. Instead, for Clustering, **Precision** measures the purity of clustering analysis results and checks how well the algorithm assigns samples of different families to different clusters; **Recall** measures the algorithm's well-grouping of malware samples from the same family.

Step1 - Hash Matching Classification: Hash-matching classification involves selecting third-party services to categorize malware based on the sample's hash. The selection process first involves evaluating third-party services, their request limit, and the information collected by querying the API, such as malware family names and other tags, and then on Precision, Recall, and Coverage metrics. After the selection process, the proposed pipeline integrates those third-party services that score high percentage values of the considered metrics.

Step2 - Yara Rules Classification: This step involves two phases. The first phase assesses the effectiveness of open-source Yara rules repositories based on their nature and malware family name rule matching percentage. The evaluation is based on the rules' Precision, Recall, and Coverage on the dataset by uploading each sample to the experimental sandbox described in Section 3. The second classification phase requires manual writing of Yara rules for malware samples still unclassified in the previous phase, such as malicious binaries and scripts. The proposed pipeline integrates those rules that score high percentage values of the considered metrics.

Step3 - Clustering: The classification methodology involves designing an algorithm based on the Precision and Recall metrics and applying it to malware families with multiple samples while discarding the remaining families. It uses Fuzzy Hashing (TLSH) to compute the TLSH digest of each file and the distance between files by using the Hamming distance for the bodies and comparing the file lengths and the quartile ratios for the headers.

Classification Aggregator: The classification aggregator uses a majority voting technique, combining multiple classifications from the matched step and labeling the malware sample with the most frequent label. Normalized classifications ensure that the final labeling aligns with the manual classification of the instances.

3 CASE STUDY

This section analyzes the results obtained by applying our approach in an industrial setting. The experiment setup involved a laptop with three virtual machines: pfSense [19] as firewall/router software distribution, the second one stored and analyzed malware samples using Karton [6] and, FLARE VM [13], a collection of Windows installation scripts for reverse engineering. The dataset was collected through an **Honeynet** and **Third party malware repositories**. In our case study, we deployed two open-source solutions, Cowrie [23] and Dionea [24] deployed in a cloud environment. These honeypots could only capture specific malware types, primarily botnets. The initial dataset included 1143 samples: 1118 captured by the honeypots between the end of July 2022 and the end of October 2022, while the others are from third-party services. For each malware family, it specifies the type of malware, the architecture, and the number of samples in the dataset. An MSc. student and a PhD student, supervised by two professors and experts in the cybersecurity field, manually labeled the known samples in the dataset to obtain a ground truth to evaluate each step of the proposed pipeline. The percentage of classified samples is 73% (837 instances), 23% (259 instances) remain unclassified, while 4% (47 instances) are corrupted, and thus, it is not possible to analyze them. Among classified samples, the top 3 most represented classes are Wannacry, Poisonedwarf, and Rapperbot Mirai Botnet. The following steps describe the results obtained using our proposed approach considering the 837 manually validated malware samples.

Step1 - Hash Matching Classification: The first classification methodology adopted the malware hash to search in a collection of appropriately filtered third-party services for classification evidence. The selection process considers 22 third-party services, selecting and integrating the following 8 in our pipeline and discarding the others. Any.run [2], VmRay [29], AlienVault [1], and ThreatFox [25] correctly classify samples, but their recall and coverage values are below 10%. Triage [26] has perfect precision and coverage of about 20%. High coverage percentages include VirusTotal [27], Metadefender [16], and HybridAnalysis [9], but their databases often assign incorrect labels.

Step2 - Yara Rules Classification: The second classification methodology analyzed uses Yara rules. By evaluating open-source Yara rules repositories, four rules repositories are incorporated into the classification process downstream of this analysis: ReversingLabs [11], InQuest [10], Yara-Rules [20], and Neo23x0 [17]. The study of 837 samples found that 70% (585 instances) were classified, and 30% (252 instances) remained unclassified. 93% (547 instances) of the classified samples are correctly classified, while 7% (38 instances) are wrongly classified. The classification was affected by the correct classification of all 531 Wannacry samples. Yara rules work with specific architectures and searches for byte sequences, leading to more matches against binaries. Some incorrectly categorized malware, such as *Unknown Mirai Botnet*, was classified by the approach, indicating that the Mirai variants share parts of

the payload. Instead, the second phase involves writing Yara rules. An MSc. student and a PhD student, supervised by two professors and experts in the cybersecurity field, wrote the rules used in this work. The process entails identifying malware key points from publications, formalizing IoC, and assessing Precision, Recall, and Coverage. In particular, we focused on unclassified or misclassified malware families through Yara rules belonging to public repositories. For **Malicious Script**, following our approach, we identified some common properties within *Undernet Perl Irc Bot* and *Ddos Perl Irc Bot* malware families, enabling the combination of the IoC of both malware families into a single Yara rule to the identification of the *Perl Irc Bot*. While, for **Malicious Binaries**, we performed several additional actions such as integrating decompression and decrypting services and reversing tools. The new rules show significant improvements in classification statistics. Our approach classified 93% (772 instances) of the 837 samples. Of these, 94% (726 instances) are correctly classified, increasing overall performance by 1%, while the 6% (46 instances) were wrongly classified. The percentage of unclassified samples is 7% (65 instances). These results are still affected by the 165 Poissondwarf samples.

Step3 - Clustering Classification: The last step uses the TLSH algorithm to create the input matrix for the DBSCAN algorithm, which groups instances into clusters based on architecture and malware family membership. The matrix uses the distance generated from the digests by comparing the concatenation of the digest body and the hexadecimal representation of the digest header for each malware sample. The results are evaluated using Precision and Recall metrics. At first, we evaluate our algorithm on the ground truth to assess the DBSCAN algorithm's performance by considering several *eps* values, the maximum distance between two samples to be in a neighborhood relationship. Its best value was 100 because it reached the highest Precision and Recall percentages. With this value, the algorithm was applied to the dataset, which consisted only of the remaining samples. The analysis highlights how that classification technique clusters only the 7% (12 instances) of the remaining samples (176 instances) while the 94% (164 instances) remains unclassified.

Classification Aggregator: The classification aggregator improves our approach performance by classifying about 95% (791 instances), of which about 99% (780 instances) are correctly classified, and about 1% (11 instances) are wrongly classified.

4 CONCLUSION AND FUTURE WORK

The study proposed a multi-step approach for malware sample classification, integrating different classification methodologies to support companies in classifying unknown malware samples. The Hash Matching classification shows powerful results but easily misclassifies malware samples due to malware component reuse. Instead, Yara rules were more accurate, but its coverage is lower than Hash Matching and Clustering. According to the cluster results, only 12 samples out of 176 are clustered. Finally, the classification aggregator's findings showed that 780 out of 791 samples were correctly classified. Only 46 out of the 837 instances remained unclassified. A critical factor is the need for authoritative solutions to validate Yara rules' depth and accuracy; nowadays, there are no solutions like that. In the future, we aim to employ larger datasets, improve the classification aggregator malware naming challenges

and automate the normalization of the results sent by different classification methodologies. In addition, we plan to integrate classification techniques based on the malware TTPs. Finally, we aim to examine the application of sigma rules to classify unknown malware samples and recognize each malware sample in a single instance.

REFERENCES

- [1] AlienVault. 2024. *AlienVault*. AlienVault. <https://otx.alienvault.com/>
- [2] Any.run. 2024. *Any.run*. Any.run. <https://any.run/>
- [3] Dmitri Bekerman, Bracha Shapira, Lior Rokach, and Ariel Bar. 2015. Unknown malware detection using network traffic classification. In *2015 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 134–142.
- [4] Gerardo Canfora, Mimmo Carapella, Andrea Del Vecchio, Laura Nardi, Antonio Pirozzi, and Corrado Aaron Visaggio. 2020. About the robustness and looseness of yara rules. In *Testing Software and Systems: 32nd IFIP WG 6.1 International Conference, ICTSS 2020, Naples, Italy, December 9–11, 2020, Proceedings 32*. Springer, 104–120.
- [5] Gerardo Canfora, Andrea Di Sorbo, Francesco Mercaldo, and Corrado Aaron Visaggio. 2015. Obfuscation techniques against signature-based detection: a case study. In *2015 Mobile systems technologies workshop (MST)*. IEEE, 21–26.
- [6] CERT-Polska. 2024. *karton*. CERT-Polska. <https://github.com/CERT-Polska/karton>
- [7] Jesús Díaz-Verdejo, Javier Muñoz-Calle, Antonio Estepa Alonso, Rafael Estepa Alonso, and Germán Madinabeitia. 2022. On the detection capabilities of signature-based intrusion detection systems in the context of web attacks. *Applied Sciences* 12, 2 (2022), 852.
- [8] Akash Garg and Prachi Maheshwari. 2016. Performance analysis of snort-based intrusion detection system. In *2016 3rd international conference on advanced computing and communication systems (icacacs)*, Vol. 1. IEEE, 1–5.
- [9] Hybrid-analysis. 2024. *Hybrid-analysis*. Hybrid-analysis. <https://www.hybrid-analysis.com/>
- [10] InQuest. 2023. *Awesome Yara*. InQuest. <https://github.com/InQuest/awesome-yara>
- [11] Reversing Labs. 2023. *Reversinglabs Yara Rules*. Reversing Labs. <https://github.com/reversinglabs/reversinglabs-yara-rules>
- [12] Adam Lockett. 2021. Assessing the Effectiveness of YARA Rules for Signature-Based Malware Detection and Classification. arXiv:2111.13910 [cs.CR] <https://arxiv.org/abs/2111.13910>
- [13] Mandiant. 2024. *flare-vm*. Mandiant. <https://github.com/mandiant/flare-vm>
- [14] Francesco Mercaldo, Andrea Di Sorbo, Corrado Aaron Visaggio, Aniello Cimitile, and Fabio Martinelli. 2018. An exploratory study on the evolution of Android malware quality. *Journal of Software: Evolution and Process* 30, 11 (2018), e1978.
- [15] Francesco Mercaldo, Vittoria Nardone, Antonella Santone, and Corrado Aaron Visaggio. 2016. Hey malware, i can find you!. In *2016 IEEE 25th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE)*. IEEE, 261–262.
- [16] Metadefender. 2024. *Metadefender*. Metadefender. <https://metadefender.opswat.com/>
- [17] Neo23x0. 2023. *signature-base*. Neo23x0. <https://github.com/Neo23x0/signature-base>
- [18] Livinus Obiora Nweke. 2021. A survey of specification-based intrusion detection techniques for cyber-physical systems.
- [19] pfSense. 2024. *pfSense*. pfSense. <https://www.pfsense.org/>
- [20] Yara Rules. 2023. *rules*. Yara Rules. <https://github.com/Yara-Rules/rules>
- [21] Igor Santos, Felix Brezo, Xabier Ugarte-Pedrero, and Pablo G Bringas. 2013. Opcode sequences as representation of executables for data-mining-based unknown malware detection. *information Sciences* 231 (2013), 64–82.
- [22] Iqbal H Sarker, ASM Kayes, Shahriar Badsha, Hamed Alqahtani, Paul Watters, and Alex Ng. 2020. Cybersecurity data science: an overview from machine learning perspective. *Journal of Big data* 7 (2020), 1–29.
- [23] Telekom Security. 2023. *Cowrie*. Telekom Security. <https://github.com/cowrie/cowrie>
- [24] Telekom Security. 2023. *Dionaea*. Telekom Security. <https://github.com/DinoTools/dionaea>
- [25] ThreatFox. 2024. *ThreatFox*. ThreatFox. <https://threatfox.abuse.ch/>
- [26] Tria.ge. 2024. *Tria.ge*. Tria.ge. <https://tria.ge/>
- [27] VirusTotal. 2024. *VirusTotal*. VirusTotal. <https://www.virustotal.com/gui/home>
- [28] Corrado Aaron Visaggio. 2023. The state of the malware: What can we defend against? In *Monitoring and Protection of Critical Infrastructure by Unmanned Systems*. IOS Press, 114–126.
- [29] Vmray. 2024. *Vmray*. Vmray. <https://www.vmray.com/>
- [30] Shen Wang and S Yu Philip. 2019. Heterogeneous graph matching networks: Application to unknown malware detection. In *2019 IEEE International Conference on Big Data (Big Data)*. IEEE, 5401–5408.